Bloco 3 - Redes e Linux

Da URL ao Pacote de Rede



Nmap observa portas; Wireshark observa trafego; Burp observa HTTP.

Bloco 3 - Redes e Linux

Aula 21 - Redes: IP, Portas, TCP e UDP

IP identifica uma interface na rede; porta identifica um servico naquela maquina. TCP cria uma conexao confiavel orientada a fluxo; UDP envia datagramas sem a mesma garantia de entrega. localhost normalmente aponta para 127.0.0.1.

Pontos-chave

- 80 HTTP, 443 HTTPS, 22 SSH e 53 DNS sao exemplos comuns.
- LISTENING significa que um processo aguarda conexoes.
- Uma mesma maquina pode ter varios servicos em portas diferentes.

Exemplo tecnico

Codigo / comando

```
const net = require("net");
const s = net.connect(3000, "127.0.0.1", () => {
  console.log("Porta acessivel"); s.end();
});
```

Portugol - equivalente didatico

```
se conectarTCP("127.0.0.1", 3000) entao
    escreva("Porta acessivel")
fimse
```

Aula 22 - Linux para Seguranca

Linux e frequente em servidores e ferramentas de seguranca. Comandos como pwd, ls, cd, cat, ps, grep e ss ajudam a navegar, ler arquivos, localizar processos e observar portas.

Pontos-chave

- Pipe | envia a saida de um comando para outro.
- Permissoes r/w/x significam leitura, escrita e execucao.
- No WSL e possivel praticar Linux dentro do Windows.

Exemplo tecnico

Codigo / comando

```
# Shell
ps aux | grep node
ss -tulpn
ls -la
```

Portugol - equivalente didatico

```
// Portugol conceitual
processos <- listarProcessos()
mostrar(filtrar(processos, "node"))
mostrar(listarPortas())
```

Aula 23 - Nmap - Mapeamento de Portas e Servicos

Nmap identifica portas abertas, fechadas ou filtradas e pode tentar reconhecer servicos. Em laboratorio, ele permite confirmar os servidores Node nas portas 3000 e 4000.

Pontos-chave

- open: servico aceitando conexao.
- closed: host respondeu, mas nada escuta ali.
- filtered: filtragem impede conclusao clara.

Exemplo tecnico

Codigo / comando

```
# Somente laboratorio/local
nmap -p 3000,4000 127.0.0.1
nmap -sV -p 3000,4000 127.0.0.1
```

Portugol - equivalente didatico

```
portas <- [3000, 4000]
para cada porta em portas faca
    escreva(testarPorta("127.0.0.1", porta))
fimpara
```

Aula 24 - Wireshark - Observando Pacotes

Wireshark captura e interpreta pacotes. Uma requisicao web passa por camadas como Ethernet, IP, TCP e HTTP/TLS. Filtros ajudam a isolar o trafego relevante.

Pontos-chave

- tcp.port == 3000 filtra a porta do laboratorio.
- http mostra trafego HTTP reconhecido.
- HTTPS cifra o conteudo da aplicacao em transito.

Exemplo tecnico

Codigo / comando

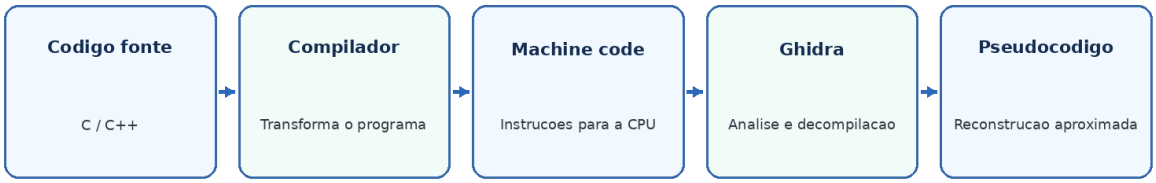
```
// Filtros Wireshark
tcp.port == 3000
http
dns
```

Portugol - equivalente didatico

```
se pacote.porta = 3000 entao
    mostrar(pacote)
fimse
```

Bloco 4 - Binarios e engenharia reversa

Do Codigo a Engenharia Reversa



O decompilador reconstrui logica, mas nao recupera perfeitamente o fonte original.

Bloco 4 - Binarios e engenharia reversa

Aula 25 - Programas Compilados, CPU e Registradores

C e compilado em instrucoes de maquina. Assembly representa essas instrucoes de forma legivel. Registradores sao pequenas areas muito rapidas dentro da CPU e participam de calculos, chamadas e controle de fluxo.

Pontos-chave

- RAX/EAX e frequentemente usado para resultados.
- RSP aponta para a stack.
- RIP indica a proxima instrucao a executar em x86-64.

Exemplo tecnico

Codigo / comando

```
// C
int x = 5;
int y = 3;
int z = x + y;
// Assembly conceitual: mov eax,5; add eax,3
```

Portugol - equivalente didatico

```
inteiro x <- 5
inteiro y <- 3
inteiro z <- x + y
escreva(z)
```

Aula 26 - Memoria: Stack, Heap e Buffer Overflow

Stack guarda chamadas e variaveis locais; heap guarda alocacoes dinamicas. Linguagens como C permitem controle direto de memoria, por isso erros de limite podem corromper memoria. Buffer overflow e estudado apenas em laboratorio controlado.

Pontos-chave

- Stack segue comportamento de pilha.
- malloc aloca no heap e free libera.
- Sempre valide tamanhos e prefira APIs seguras.

Exemplo tecnico

Codigo / comando

```
char nome[8];
// forma segura: limitar a copia ao tamanho do buffer
snprintf(nome, sizeof(nome), "%s", entrada);
```

Portugol - equivalente didatico

```
cadeia nome
leia(entrada)
se comprimento(entrada) <= 7 entao
    nome <- entrada
senao
    escreva("Entrada grande demais")
fimse
```

Aula 27 - Assembly - Comparacoes e Saltos

Assembly mostra como if, loops e chamadas viram comparacoes e saltos. cmp compara valores e instrucoes como je/jne alteram o fluxo conforme flags da CPU.

Pontos-chave

- mov move/carrega valores.
- add/sub realizam aritmetica.
- cmp + jump implementam decisoes.

Exemplo tecnico

Codigo / comando

```
; Assembly conceitual
cmp eax, 10
je igual
mov ebx, 0
jmp fim
igual:
mov ebx, 1
fim:
```

Portugol - equivalente didatico

```
se x = 10 entao
    resultado <- 1
senao
    resultado <- 0
fimse
```

Aula 28 - Ghidra e Engenharia Reversa

Ghidra analisa binarios e tenta reconstruir funcoes, strings, referencias e pseudocodigo. Decompilacao e aproximada: nomes de variaveis e estruturas originais podem ter sido perdidos.

Pontos-chave

- Analise estatica nao executa o binario.
- Strings e imports oferecem pistas.
- Compare pseudocodigo com assembly quando algo parecer estranho.

Exemplo tecnico

Codigo / comando

```
// Fonte hipotetico
int validar(int n) {
  return n == 1234;
}
// Ghidra pode reconstruir logica semelhante.
```

Portugol - equivalente didatico

```
funcao logico validar(inteiro n)
    retorne n = 1234
fimfuncao
```

Aula 29 - Malware - Conceitos e Analise Segura

Malware e software com comportamento malicioso. Analise defensiva comeca por hash, strings, metadados, imports e engenharia reversa antes de qualquer execucao. Ransomware, spyware, worm e trojan descrevem familias/comportamentos diferentes.

Pontos-chave

- Nao execute arquivo suspeito no computador comum.
- Analise estatica primeiro.
- Sandbox isolada e usada apenas quando a analise dinamica e necessaria.

Exemplo tecnico

Codigo / comando

```
const crypto = require("crypto");
const hash = crypto.createHash("sha256").update(buffer).digest("hex");
console.log(hash);
```

Portugol - equivalente didatico

```
hash <- calcularSHA256(arquivo)
escreva(hash)
```

Aula 30 - Stuxnet e Sistemas Industriais

Stuxnet e um caso historico de malware voltado a ambiente industrial. O ponto didatico e entender a cadeia digital-fisica: Windows, software industrial, PLCs, processo e telemetria. Aqui estudamos arquitetura e impacto, nao codigo operacional de malware.

Pontos-chave

- PLC controla processos fisicos.
- Ataques ICS podem alterar processo e observacao.
- Seguranca industrial exige conhecimento de TI e engenharia.

Exemplo tecnico

Codigo / comando

```
// Modelo defensivo de monitoramento
if (Math.abs(sensor - referencia) > limite) {
  gerarAlerta();
}
```

Portugol - equivalente didatico

```
se valorAbsoluto(sensor - referencia) > limite entao
    gerarAlerta()
fimse
```